map-charting-student-math-misunderstandings

10th Place Solution

Rank	10
Team	Liwen Zheng
Author	Levin
Topic ID	612038
Version	Original

Source: https://www.kaggle.com/competitions/map-charting-student-math-misunderstandings/discussion/612038
Retrieved with Kaggle CLI on 2026-10-21.

Huge thanks to the organizers and Kaggle — my first medal-focused entry, and my first-ever medal: a solo gold 🥇. Below is a concise summary of the approach.

Overview

Treat the task as direct multi-class classification and ensemble 15 QLoRA-finetuned models (5-fold × 3 backbones: Qwen3-Reranker-8B, Qwen3-Embedding-8B, Qwen2.5-32B-Instruct).

Key ingredients

- Data hygiene & fold control: auto-fix True/False inconsistencies; keep near-duplicates in the same fold.
- Focal + CE mixed loss with class-weight warmup (~33%) to stabilize long-tail classes.
- Inference acceleration: for Qwen2.5-32B-Instruct, merge LoRA → split base/cls-head → GPTQ-4bit base + FP16 classifier head → vLLM embed → logits → ensemble.

Models & Training

Backbones (all via `AutoModelForSequenceClassification`)

1. Qwen3-Reranker-8B (QLoRA)
2. Qwen3-Embedding-8B (QLoRA)
3. Qwen2.5-32B-Instruct (QLoRA)

Each backbone is trained with 5 folds ⇒ 15 models total.

Loss & optimization

- Loss: a mixture of Focal (computed on unsmoothed CE to emphasize hard examples) and Cross-Entropy with label smoothing = 0.1 for calibration; per-class weights applied after warmup.
- Warmup: `weight_warmup_ratio ≈ 0.33` of total steps to avoid unstable early updates on the long tail.
- Quant-aware training: the 8B models use bitsandbytes 4-bit NF4 + gradient checkpointing (QLoRA-style).

Hyperparameters

- Qwen3-Reranker-8B / Qwen3-Embedding-8B (single RTX 4090)
  `lr=1e-4`, `epochs=4`, scheduler=`cosine`, `weight_warmup_ratio=0.33`, effective global batch size=16.
- Qwen2.5-32B-Instruct (A800-40GB)
  `lr=1e-4`, `epochs=3`, `weight_warmup_ratio=0.33`, effective global batch size=32.

LoRA config (typical): `r=64, alpha=128, dropout=0.05`, target Q/K/V/O and MLP projections; classifier head (`score`) is saved.

Inference Engineering

8B models

- Do not merge LoRA. Save a 4-bit quantized base and load it with the LoRA adapter for inference so a single GPU can handle each model efficiently (also friendly to dual-T4 scheduling).

32B model

- Workflow: merge LoRA → split base & classifier head → GPTQ-4bit quantize the base (keep cls-head in FP16) → run vLLM in `embed` mode → apply an external Linear head to produce logits → write logits.

```
():

   ()
   ()
   os.makedirs(args.base_model_save_path, exist_ok=)
   os.makedirs(args.classifier_head_save_path, exist_ok=)

   full_model = AutoModelForSequenceClassification.from_pretrained(
       args.merged_model_path,
       device_map=,
       torch_dtype=torch.float16
   )
   tokenizer = AutoTokenizer.from_pretrained(args.merged_model_path)

   base_model = full_model.model
   classifier_head = full_model.score

   ()
   base_model.save_pretrained(args.base_model_save_path)
   tokenizer.save_pretrained(args.base_model_save_path)

   ()
   torch.save(classifier_head.state_dict(), os.path.join(args.classifier_head_save_path, ))

   classifier_config = {
       : full_model.config.hidden_size,
       : full_model.config.num_labels,
       : full_model.config.id2label,
       : full_model.config.label2id,
       : classifier_head.bias   ,
   }
    (os.path.join(args.classifier_head_save_path, ), )  f:
       json.dump(classifier_config, f)

    shutil
   shutil.rmtree(args.merged_model_path)
   ()

   ()

():
    vllm  LLM

   ()
   ()
   ()

    (os.path.join(args.classifier_head_path, ), )  f:
       classifier_config = json.load(f)
```

```
    classifier_head = nn.Linear(
        classifier_config[],
        classifier_config[],
        bias=(classifier_config.get(, ))
    )
    state = torch.load(os.path.join(args.classifier_head_path, ), map_location=)
    classifier_head.load_state_dict(state)
    classifier_head.to().()

    llm = LLM(
        model=args.quantized_model_path,
        quantization=,
        dtype=,
        task=,
        override_pooler_config={: , : },
        tensor_parallel_size=
    )
    ()

    le = LabelEncoder()
    train_df = pd.read_csv(args.label_data_path)
    train_df.Misconception = train_df.Misconception.fillna()
    train_df[] = train_df.Category +  + train_df.Misconception
    le.fit(train_df[])

    all_results = []
    :
        embed_outputs = llm.embed(TEST_PROMPTS)
        embeddings = torch.from_numpy(np.stack([np.array(o.outputs.embedding, dtype=np.float32)  o
embed_outputs])).to()

         torch.no_grad():
            logits = classifier_head(embeddings)
            probs = torch.softmax(logits, dim=-).cpu().numpy()
            top_indices = np.argsort(-probs, axis=)

         i  ((TEST_PROMPTS)):
            pred_id = top_indices[i, ]
            top_3_ids = top_indices[i, :]
            result = {
                : i,
                : TEST_PROMPTS[i],
                : le.inverse_transform([pred_id])[],
                : (probs[i, pred_id]),
                : le.inverse_transform(top_3_ids).tolist(),
                : probs[i, top_3_ids].tolist()
            }
            all_results.append(result)
            ()

    :
         llm, classifier_head; gc.collect(); torch.cuda.empty_cache()

     (args.output_json_path, )  f:
        json.dump(all_results, f, indent=)
    ()
    ()
```

- Quantization: GPTQ-4bit with `group_size=64`, symmetric, GAR (Group-Aware Reordering), `true_sequential=True`; ~2k calibration texts formatted like training prompts.

- vLLM settings:
  `quantization="gptq"`, `dtype="half"`, `task="embed"`,
  `override_pooler_config={"pooling_type": "LAST", "normalize": False}`.

Ensembling

I used a weighted logit blend—weight 2 for each 32B fold and 1 for each 8B fold—then took Top-3 for MAP@3, which proved more stable than probability averaging.

What Didn’t Work (for me)

- Hierarchical multi-task classifier with hard logic constraints
- 32B model as pointwise reranker
- 32B model as listwise reranker
- TTA

Given the label noise, long tail, and MAP@3 objective, clean splits + balanced training + many modest models beat heavier pipelines.

Results

Setting	Public MAP@3	Private MAP@3
Single 8B model	0.946	0.942
8B backbones × 5 folds	0.950	0.946
32B-Instruct × 5 folds	0.950	0.947
Ensemble (15 models)	0.951	**0.948**

Acknowledgments & References

Many thanks to the Kaggle community for the discussions, notebooks, and write-ups that informed my data hygiene, training, quantization, and ensembling choices. In particular:

- brendanartley — discussion
- kishanvavdara — notebook: Ensemble Gemma/Qwen/DeepSeek
- cdeotte — notebook: Gemma2-9B-IT CV 0.945
- jaytonde — notebook: DeepSeekMath-7B LB 0.944
- ebinan92 — 5th-place solution write-up

- yannan90 — 8th-place solution write-up
- sayoulala — discussion
- nikitababich — 6th-place solution write-up
- conjuring92 — 1st-place detailed solution)

Supplemental Q&A

common-people · 2025-10-16T11:53:56.307000

Thanks for sharing.
I have some points:
How is these three models work, And how is the training data (if some ways must be taken to balance data bias or else problems)for the reranker model and is it pairwise or listwise. looking for your explaination, thank you
Qwen3-Reranker-8B (QLoRA)
Qwen3-Embedding-8B (QLoRA)
Qwen2.5-32B-Instruct (QLoRA)

Levin · 2025-10-16T13:47:36.587000

All three models are classification models loaded using AutoModelForSequenceClassification, just like cdeotte — notebook: Gemma2-9B-IT CV 0.945

For the pairwise and listwise rerankers, I referred to ebinan92 — 5th-place solution write-up.

Specifically, for the listwise reranker, I first took the top 5 candidates from the prediction results of each sample generated by the recall models. Then, I concatenated some information (such as explanations for the top 5 candidates and all candidate answers for the question) with the prompt. Finally, I fed this combined input into AutoModelForCausalLM to have the model output options A, B, C, D, and E.

However, this approach did not yield ideal results. For me, the best performance using the 14B 5-fold model was 0.948 in the Public dataset and 0.945 in the Private dataset. Maybe my method was wrong; in any case, I finally gave up on the reranker approach.

Bilzard · 2025-10-17T07:58:19.390000

@zhengliwen Congratulations for solo gold.

I'm interested in handling with rare classes.

> Loss: a mixture of Focal (computed on unsmoothed CE to emphasize hard examples) and Cross-Entropy with label smoothing = 0.1 for calibration; per-class weights applied after warmup.
> Warmup: weight_warmup_ratio ≈ 0.33 of total steps to avoid unstable early updates on the long tail.

1. Do you use isolated warmup parameter `weight_warmup_ratio` in addition to the `warmup_ratio`?
2. Do per-class weights means (alpha, gamma) per each class? How these parameters are chosen, manually? If so, do you mind sharing these parameters?

$$
\mathrm{FL}(p_t) = - \alpha_t (1 - p_t)^{\gamma} \log(p_t)
$$

Levin · 2025-10-17T13:15:54.730000

In addition to the main warmup, I use a separate weight warmup (~33% of total steps).

```
():
    ():
       ().__init__(*args, **kwargs)
       .combined_loss_fn = combined_loss_fn
       .accelerator = accelerator

    ():
       labels = inputs.get()
        .combined_loss_fn     (.combined_loss_fn, ):

           :
               current_step = .state.global_step
            Exception:
               current_step =
           .combined_loss_fn.set_step(current_step)
       labels = inputs.pop(, )
       outputs = model(**inputs)
       logits = outputs.logits

        .combined_loss_fn   :
           loss_dict = .combined_loss_fn(logits, labels)
            (loss_dict, ):
               loss = loss_dict[]
                .accelerator    .accelerator.is_main_process:
                   wandb.log({
                       : loss_dict[].item(),
                       : loss_dict[].item(),
                       : loss.item()
                   })
           :
               loss = loss_dict
       :
           loss = outputs.loss

        (loss, outputs)  return_outputs  loss

(nn.Module):
    ():
       ().__init__()
       .focal_loss = FocalLoss(
           alpha=alpha,
           gamma=gamma,
       )
       .ce_loss_fn = nn.CrossEntropyLoss(label_smoothing=label_smoothing, reduction=)
       .focal_weight = focal_weight
       .ce_weight = ce_weight
       .class_weights = class_weights
       .weight_warmup_steps = weight_warmup_steps
       .current_step =

    ():
       .current_step = step
       .focal_loss.set_step(step)

    ():
```

```
          .class_weights    .current_step < .weight_warmup_steps:

          .class_weights

      ():
         effective_weights = .get_effective_weights()
          effective_weights   :
             effective_weights = effective_weights.to(inputs.device)

         focal = .focal_loss(inputs, targets)

         ce_loss = .ce_loss_fn(inputs, targets)
          effective_weights   :
             class_weight_per_sample = effective_weights[targets]
             ce_loss = ce_loss * class_weight_per_sample
         ce = ce_loss.mean()

         combined = .focal_weight * focal + .ce_weight * ce

          {
             : combined,
             : focal.detach(),
             : ce.detach(),
             : effective_weights
         }

  (nn.Module):
      ():
         ().__init__()
         .alpha = alpha
         .gamma = gamma

      ():
         ce = F.cross_entropy(inputs, targets, reduction=)
         pt = torch.exp(-ce)
         focal_factor = ( - pt) ** .gamma

         focal_loss = .alpha * focal_factor * ce
         focal_loss = focal_loss.mean()

          focal_loss


combined_loss_fn = CombinedLoss(
         focal_weight=args.focal_weight,
         ce_weight=-args.focal_weight,
         alpha=,
         gamma=,
         label_smoothing=args.label_smoothing,
         num_classes=n_classes,
         class_weights=class_weights_tensor,
         weight_warmup_steps=WEIGHT_WARMUP_STEPS,
     )
```

common-people · 2025-10-17T13:25:24.177000

Oh, thanks, so is the 8B reranker the pairwise. So you choose the kfold recall models to generate the top2 training data of it? I also tried the listwise reranker approch which was not work in my code

!!!!! · 2025-10-17T16:42:26.720000

Congratulations ! May I ask how long it takes to train a 32B model? If the effective global batch size is 32, does that mean gradient accumulation was used?

Levin · 2025-10-18T01:50:46.590000

Thanks! For my runs (3 epochs, effective global batch = **32**):
2× L20 40GB
Per-GPU batch = **16**, **no gradient accumulation** (GA=1) → **~5 hours**.
2× A100 40GB
Same settings (per-GPU batch 16, GA=1) → **~3 hours**.
4× A100 40GB
Per-GPU batch = **8**, **no gradient accumulation** (8×4=32, GA=1) → **~2 hours**.

Levin · 2025-10-18T06:32:26.153000

I use k-fold recall models to produce the top-5 candidates and train a listwise reranker on them; it still didn't outperform a strong classifier.
If yours isn't working, double-check (1) leakage across folds and (2) next-token alignment (choose the correct label token).

Gaurav Rawat · 2025-10-18T13:09:30.760000

Also how do you decide between changing to hybrid loss functions I find it works sometimes but do you have any reasoning when change loss functions or its just more trial and error to what works on cv

Levin · 2025-10-18T17:01:04.567000

Thanks! Pretty much CV first. Start with CE (+ smoothing), nudge things a bit, and keep what improves

Roschild.Rui · 2025-10-20T02:29:42.473000

@zhengliwen congratulation!
May i ask which version of vllm are you using? Did you make some modify to source code to support 14b quantization model infer in emb mode in T4?

Levin · 2025-10-20T05:03:29.750000

No source mods needed — just pip install vllm. I'm using the standard pip release (no local patches). vLLM's embedding API works out of the box for supported decoder-only models.

You can check the supported list here (LLM.embed):
https://docs.vllm.ai/en/latest/models/supported_models.html#embedding

Roschild.Rui · 2025-10-22T06:20:12.470000

Thanks! which specific vllm version do you use? We met some problems when setting `task=emb` and using 2 gpus parallel inference with vllm 0.11.0

Levin · 2025-10-22T08:36:49.440000

Thanks! I'm also on vLLM 0.11.0 (pip) in the notebook and task="embed" works for me.

For 2×GPU, a few things to double-check:

- Set tensor_parallel_size=2.
- If your backbone is GPTQ-4bit, pass quantization="gptq" and a pooler (e.g. override_pooler_config={"pooling_type":"LAST","normalize":False}).

- Make sure the model is in the supported list for LLM.embed.